

朱小影,左正康. 日程冲突检测算法建模及自动化验证方法 [J]. 江西师范大学学报(自然科学版), 2026, 50(1): 43-48.

ZHU Xiaoying, ZUO Zhengkang. The modeling and automated verification method for schedule conflict detection algorithm [J]. Journal of Jiangxi Normal University (Natural Science Edition), 2026, 50(1): 43-48.

文章编号: 1000-5862(2026)01-0043-06

日程冲突检测算法建模及自动化验证方法

朱小影,左正康*

(江西师范大学人工智能学院, 江西 南昌 330022)

摘要: 日程冲突检测算法在会议管理、教学排课和医疗排班等场景中具有重要价值,其核心问题可抽象为时间区间的重叠检测。区间树作为典型的区间查询结构,能够高效支持插入、删除与查询操作,因此区间树成为实现该类算法的理想选择。为确保算法的可靠性与可扩展性,该文基于区间树构建了函数式建模库,并完善了配套的验证引理库,以保证其基本操作在严格形式化语义下的正确性。在此基础上,进一步完成了日程冲突检测算法的形式化建模与自动化验证实验,依托所构建的建模库与验证引理库,算法的功能与结构正确性能够借助自动化证明工具完成验证,从而有效地降低了人工证明成本,验证了所给方法的可行性与有效性。

关键词: 日程冲突检测; 区间树; Isabelle 定理证明器; 函数式建模; 自动化验证

中图分类号: TP 311 **文献标志码:** A **DOI:** 10.16357/j.cnki.issn1000-5862.2026.01.05

0 引言

在会议管理、教学排课、医疗排班等多种实际应用中,日程冲突检测算法扮演着关键角色。随着信息化管理系统的普及,数据规模和调度复杂度不断增长,时间资源冲突带来的影响也愈发显著。如何在海量动态数据中快速、准确地检测潜在冲突已成为提升现代调度系统性能与可靠性的重要课题^[1]。传统的软件开发流程通常依赖经验性测试来保障算法的可靠性,但测试方法难以覆盖所有可能的输入与边界情况,尤其在涉及区间重叠判断和复杂调度约束时,更容易产生潜在漏洞。相比之下,形式化方法能够在严谨的数学逻辑框架下对算法进行建模与推理,从而在更高层次上保证其正确性^[2-3]。Isabelle/HOL^[4-5]是一种成熟的交互式定理证明器,提供了强大的高阶逻辑建模能力,支持定义复杂的数据结构、函数以及关系,并能在严格的逻辑体系下对它们进行推理与证明。而且,Isabelle

提供丰富的自动化工具,如 simp 简化器、auto^[6] 推理器和 Sledgehammer^[7] 自动证明助手等,可在用户指定的范围内自动搜索证明步骤,从而显著地降低人工干预的工作量。因此,将 Isabelle 作为日程冲突检测算法及其底层区间树结构的验证工具是实现高可靠性算法设计与验证的有效路径。

目前,有关冲突检测算法的研究主要聚焦于工程实现与性能优化,强调提升系统的运行效率与响应能力。文献^[8]提出基于任务网络的动态排程算法,通过启发式方法调整任务顺序以避免冲突;文献^[9]在时间敏感网络中设计高效流冲突检测机制,通过路径预处理减少冗余查询并优化调度策略;针对在线上排程中的时间区间冲突问题,文献^[10]提出了决策算法,通过筛选与淘汰冲突项目来达成最终排程。此外,基于 XACML 的访问控制策略冲突检测方法^[11]和在 CALO 智能助手系统中的多代理日程协调机制^[12]也在各自领域中取得应用成效。然而,这些方法普遍缺乏形式化语义描述与正确性验证,算法可靠性依赖实验测试,难以从理论

收稿日期: 2025-07-12

基金项目: 国家自然科学基金(62462036)和江西省自然科学基金(20242BAB26017)资助项目。

通信作者: 左正康(1980—),男,江西抚州人,教授,博士,博士生导师,主要从事高可信与智能化软件研究。E-mail: zuo803@jxnu.edu.cn

层面得到保证. 为此, 引入形式化建模与自动化验证框架, 对日程冲突检测算法进行语义级验证, 已成为提升算法可信性与可复用性的有效途径.

本文提出了日程冲突检测算法的函数式建模与自动化验证方法. 通过构建以区间树为核心的函数式建模库, 对插入、删除、查询等核心操作进行了定义. 进一步地, 建立了配套的验证引理库, 用以分别验证这些操作的功能正确性与结构正确性, 从理论层面确保算法在所有输入条件下的可靠性. 在此基础上, 将日程冲突检测算法抽象为区间重叠检测问题, 并通过复用区间树的函数式建模与验证结果实现了插入与删除操作的自动化验证, 显著地降低了人工证明的复杂度.

1 背景知识

1.1 区域(locale)

在 Isabelle 中, 区域(locale) 是一种重要的理论组织机制, 它提供了模块化机制, 用于在形式化建模中引入一个带有上下文假设的工作环境^[12]. 区域不仅能够定义常量、函数和谓词, 还可以附加一定的前提条件或公理, 从而形成一个局部的理论框架. 在 Isabelle 中, locale 的定义通常采用如下结构:

```
locale loc_name =
  fixes parameter1::type1
  fixes parameter2::type2
  ...
  assumes logical_spec1:  $\psi_1$ 
  assumes logical_spec2:  $\psi_2$ 
  ...
```

locale 的声明一般由 3 个部分构成: 1) 区域名称, 用于标识该 locale, 为后续调用和实例化提供统一的引用; 2) 局部参数(使用 fixes 关键字), 定义参数及其类型, 在区域内部引入抽象常量或运算符, 相当于为模块设定接口参数, 使区域具备参数化特性; 3) 逻辑规约(使用 assumes 关键字), 为局部参数附加逻辑约束或公理条件, 用于限定可接受的实例范围, 并确保区域内部定理的推导在这些前提下有效.

1.2 区间树

区间树(interval tree)^[13-14] 是一种用于处理区间数据的平衡二叉搜索树结构. 它的核心思想是: 在树的每个结点中存储一个区间, 同时维护额外的信息(通常是在该结点子树中所有区间右端点的最大值). 这种结构的设计使得区间树能够高效支持区

间重叠检测、区间查询及动态更新等操作.

在函数式建模的语义下, 区间树可被抽象表示为一种通用的树结构, 其节点由区间及其附属信息组成^[15], 具体定义如下:

```
type_synonym 'a ivl_tree = "( 'a ivl * 'a)
tree".
```

该定义表明, 区间树('a ivl_tree) 是一种以区间('a ivl) 为关键字、以区间最大端点值('a) 为节点附属信息的函数式二叉树结构.

由于区间树在高效区间查询与动态更新方面具有优越性能, 所以区间树被广泛应用于多种时间或空间区间管理的场景^[16-17], 如日程冲突检测、数据库区间索引、计算几何问题求解以及操作系统内存区间分配等.

2 区间树函数式建模库和验证引理库

2.1 函数式建模库

为支持日程冲突检测算法的形式化验证, 构建了一个基于区间树的函数式建模库. 该建模库采用纯函数式范式, 避免了可变状态与副作用对语义带来的不确定性, 从而能够在 Isabelle/HOL 的高阶逻辑框架下对数据结构和算法操作进行准确刻画. 通过函数式抽象, 将区间树的构造与操作以简洁、统一的形式表达, 显著降低了复杂数据结构建模与验证的难度. 该建模库共实现了 16 种操作(见图 1), 涵盖了区间树的查询类函数、转换类函数、修改类函数及不变式函数, 下面将具体介绍这四类函数.

1) 查询类函数. 查询类函数主要用于对在树结构中的区间信息进行检索与判断, 包含的函数有查找、查找最小/最大区间、前继、选择、树的大小、树高、区间重叠判断.

2) 转换类函数. 转换类函数用于将区间树转换成不同的数据表示形式, 包括中序、先序和后序遍历函数, 将树按不同顺序展开为列表; 同时定义集合化函数, 将树中所有区间转换为集合形式.

3) 修改类函数. 修改类函数主要负责对区间树结构与节点内容进行调整, 以支持区间数据的插入、删除操作. 其核心是利用辅助函数对区间上限最大值的维护, 以确保各节点始终满足区间上限最大值不变式.

4) 不变式函数. 不变式函数是指在数据结构的整个生命周期中始终保持为真的性质或约束条件. 它反映了结构内部的逻辑一致性和正确性, 是确保

数据结构在各种操作后依然处于合法状态的核心依据^[18]. 区间树的不变式需要保证节点区间上限的最大值正确以及节点中序遍历有序性.

	locale Interval_Tree =
空树	fixes empty :: "t"
查找	fixes isin :: "'a::linorder ⇒ t ⇒ bool"
最小值	fixes min :: "t ⇒ 'a"
最大值	fixes max :: "t ⇒ 'a"
树的大小	fixes size :: "t ⇒ nat"
树高	fixes height :: "t ⇒ nat"
前继	fixes pre :: "'a ⇒ t ⇒ 'a option"
查找第k小	fixes select :: "t ⇒ nat ⇒ 'a option"
判断区间重叠	fixes search :: "t ⇒ 'a ivl ⇒ bool"
修改类函数	插入 fixes insert :: "'a ⇒ t ⇒ t"
	删除 fixes delete :: "'a ⇒ t ⇒ t"
转换类函数	中序遍历 fixes inorder :: "t ⇒ 'a list"
	前序遍历 fixes preorder :: "t ⇒ 'a list"
	后序遍历 fixes postorder :: "t ⇒ 'a list"
不变式函数	集合化 fixes set_tree :: "t ⇒ 'a set"
	不变式 fixes invar :: "t ⇒ bool"

图 1 区间树区域

2.2 验证引理库

为保障 16 种区间树操作在功能上与结构上都

空树的功能正确性	assumes set_empty: "set_tree empty = {}"
查找的功能正确性	assumes set_isin: "invar t ⇒ isin x t = list_isin x (inorder t)"
最小值的功能正确性	assumes min_list: "invar t ⇒ t ≠ empty ⇒ min t = list_min (inorder t)"
最大值的功能正确性	assumes max_list: "invar t ⇒ t ≠ empty ⇒ max t = list_max (inorder t)"
树的大小的功能正确性	assumes size_list: "invar t ⇒ size t = length (inorder t)"
树高的功能正确性	assumes height_length: "invar t ⇒ height t ≤ length (inorder t)"
前继的功能正确性	assumes pre_list: "invar t ⇒ pre a t = list_pred a (inorder t)"
查找第k小的功能正确性	assumes select_list: "invar t ⇒ k > 0 ⇒ select t k = list_sel (k-1) (inorder t)"
判断区间重叠的功能正确性	assumes search_set: "invar t ⇒ search t x = has_overlap (set_tree t) x"
插入的功能正确性	assumes set_insert: "invar t ⇒ inorder(insert x t) = ins_list x (inorder t)"
删除的功能正确性	assumes set_delete: "invar t ⇒ inorder(delete x t) = del_list x (inorder t)"
中序遍历的功能正确性	assumes inorder_set: "set (inorder t) = set_tree t"
前序遍历的功能正确性	assumes preorder_set: "set (preorder t) = set_tree t"
后序遍历的功能正确性	assumes postorder_set: "set (postorder t) = set_tree t"

图 2 基础操作的功能正确性逻辑规约

2.2.2 结构正确性引理 结构正确性引理主要用于验证在修改类函数执行插入、删除操作后数据结构仍然满足区间树的定义与逻辑约束,从而确保树的结构稳定性与可维护性. 这一部分的验证以不变式函数为核心,通过形式化方式来保证修改操作不会破坏区间树的结构性质.

通过对每个修改类函数的形式化证明,验证引理库确保它满足:若修改前区间树满足所有不变式,则在执行插入或删除操作后,新的树结构依然保持这些性质不变. 换言之,每一次修改都不会引入结构性错误或逻辑矛盾. 总体而言,结构正确性引理的建立使得区间树在反复更新与动态维护过程中始终保持一致的逻辑结构和有序性,为算法的

满足规范,本文构建了一个系统化的验证引理库. 该引理库的核心目标包括 2 个方面: 1) 证明每个操作的功能正确性,即操作按照规范产生预期的语义结果; 2) 证明操作对区间树的结构性质的保持. 引理库按性质与用途可分为两类,即功能正确性引理和结构正确性引理,每类包含若干可复用的引理和辅助引理.

2.2.1 功能正确性引理 功能正确性引理是验证体系的核心部分,针对区间树定义的基本操作逐一建立形式化证明. 查询类函数对应的引理验证了这些函数在满足区间树结构约束的前提下能够返回与输入数据一致的逻辑结果,即查询结果的正确性与完备性. 转换类函数确保在转换过程中不会遗漏或重复节点. 修改类函数负责对区间树进行插入、删除操作,验证工作主要集中在操作执行后的修改结果的正确性,即经插入或删除操作后树中包含的区间列表与预期一致. 各个基础操作的功能正确性逻辑规约如图 2 所示.

可靠执行提供了关键的理论支撑. 修改类函数的结构正确性的逻辑规约分 2 种情况.

插入的结构正确性: assumes invar_insert: "invar t ⇒ iavar (insert x t)".

删除的结构正确性: assumes invar_delete: "invar t ⇒ invar (delete x t)".

3 日程冲突检测算法建模、自动化验证

3.1 日程冲突检测算法建模

根据日程冲突检测算法的形式化规约,其插入日程操作可用霍尔三元组加以刻画:

$$\{ \text{invar } t \} (b, t') := \text{insert_calendar } x \ t \{ (b = \text{True} \wedge (\text{inorder } t' = \text{ins_list } x (\text{inorder } t)) \wedge \text{invar } t') \vee (b = \text{False} \wedge t' = t) \}.$$

该规约精确描述了当在区间树上执行插入操作时算法的功能行为和正确性约束. 具体而言, 在一棵有效的区间树上, 当向区间树 t 中插入新区间 x 时, 若插入成功 ($b = \text{True}$), 则生成的新区间树 t' 的中序遍历结果应与在原树 t 的中序序列中插入区间 x 所得的序列完全一致, 这表明插入操作在逻辑上等价于在有序列表中插入新区间. 同时, t' 仍需满足区间树的不变式, 确保其结构正确性与有序性. 若插入失败 ($b = \text{False}$), 则检测到区间冲突, 此时算法不对原树进行任何修改, 即保持 $t' = t$, 以保证区间树的结构稳定与数据一致性.

删除日程操作可用如下形式加以刻画:

$$\{ \text{invar } t \} (b, t') := \text{delete_calendar } x \ t \{ (b = \text{True} \wedge (\text{inorder } t' = \text{del_list } x (\text{inorder } t)) \wedge \text{invar } t') \vee (b = \text{False} \wedge t' = t) \}.$$

类似于插入操作, 删除日程操作同样在区间树不变式成立的前提下进行刻画. 对于一棵满足不变式的区间树 t , 当执行删除操作 $\text{delete_calendar } x \ t$ 时, 若待删除的区间 x 存在于树中, 则删除成功 ($b = \text{True}$), 此时得到的新树 t' 的中序遍历结果应等价于从原树 t 中序序列中删除区间 x 所得到的列表, 即 $\text{inorder } t' = \text{del_list } x (\text{inorder } t)$, 同时新树 t' 仍需满足区间树的不变式. 若区间 x 不存在于原树中, 则删除操作失败 ($b = \text{False}$), 算法不对原区间树进行任何修改, 即保持 $t' = t$.

使用包的引入机制, 实现模块间程序的复用^[9], 区间树模块的复用如图3所示. 将区间树相关文件通过 `imports` 命令导入当前文件中以进行复用, 使不同算法模块在共享底层数据结构与验证库的基础上保持逻辑隔离与独立验证能力. 该机制不仅减少了冗余代码与重复建模工作, 还保证了在复用区间树模块时相关性质与不变量可自动继承, 从而提升了整体建模与验证过程的可维护性与一致性.

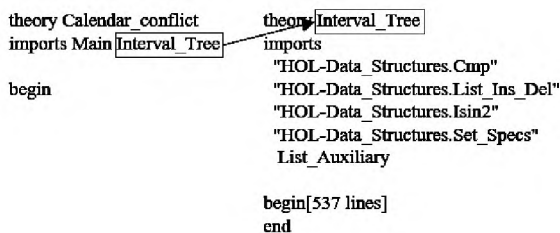


图3 包的引入机制

根据在函数式建模库中已有函数, 可直接调用

作为中间函数, 从而简化建模过程. 其中, 判断区间是否重叠函数 (`search`)、插入函数 (`insert`)、查找函数 (`isin`)、删除操作 (`delete`) 均已在建模库中定义并验证, 无须二次定义, 均可直接复用. 在此基础上, 实现日程冲突检测算法的日程插入、删除操作, 使算法建模更加简洁清晰, 具体实现如下:

//插入日程

```

definition insert_calendar::" 'a:: { linorder, order_bot } ivl => 'a ivl_tree => bool * 'a ivl_tree"
  where
    " insert_calendar ivl t =
      ( if ( search ivl ) then ( False, t ) else ( True, insert ivl t ) ) ".

```

//删除日程

```

definition delete_calendar::" 'a:: { linorder, order_bot } ivl => 'a ivl_tree => bool * 'a ivl_tree"
  where
    " delete_calendar ivl t =
      ( if isin ivl t then ( True, delete ivl t ) else ( False, t ) ) ".

```

`insert_calendar` 函数用于向区间树中插入新的日程. 它首先调用 `search` 函数检查待插入区间是否与已有区间重叠, 若存在冲突, 则返回 (`False, t`) 并保持原树不变; 若没有冲突, 则调用 `insert` 函数将区间插入树中, 并返回 (`True, insert ivl t`). 该函数的主要功能是在保证日程不冲突的前提下安全地添加新日程, 同时维护区间树的结构完整性. `delete_calendar` 函数用于从区间树中删除指定的日程. 它通过 `isin` 函数判断区间是否存在于树中, 若存在则调用 `delete` 函数删除该区间并返回 (`True, delete ivl t`); 若不存在则返回 (`False, t`). 该函数的功能是确保只删除实际存在的日程, 避免误操作, 并保持区间树的正确性和稳定性.

3.2 日程冲突检测算法自动化验证

在形式化验证过程中, 日程冲突检测算法的正确性验证依赖于已构建的验证引理库. 该引理库涵盖了区间树及其相关操作的基本性质与函数正确性证明, 其中包括 `search`、`insert`、`isin` 和 `delete` 核心函数的验证结果. 这些基础性定理构成了上层算法的复用与验证的逻辑依托.

在此基础上, 日程冲突检测算法的验证过程主要通过对引理库中相关函数与不变式的复用来实现. 由于底层操作的语义和性质均已形式化刻画, 所以验证任务可聚焦于算法整体的功能正确性与

状态保持性. 为了提高验证的可行性与自动化程度, 首先对主定理进行适当拆解, 将复杂的验证目标分解为若干独立子引理, 每个子引理对应算法中的关键操作或状态转换. 借助 Isabelle/HOL 提供

的自动化证明工具、auto 方法与 try 命令, 系统能够在引理库支持下自动完成证明过程, 从而显著提升验证效率并减少人工干预. 日程冲突检测算法自动化验证过程如图 4 所示.

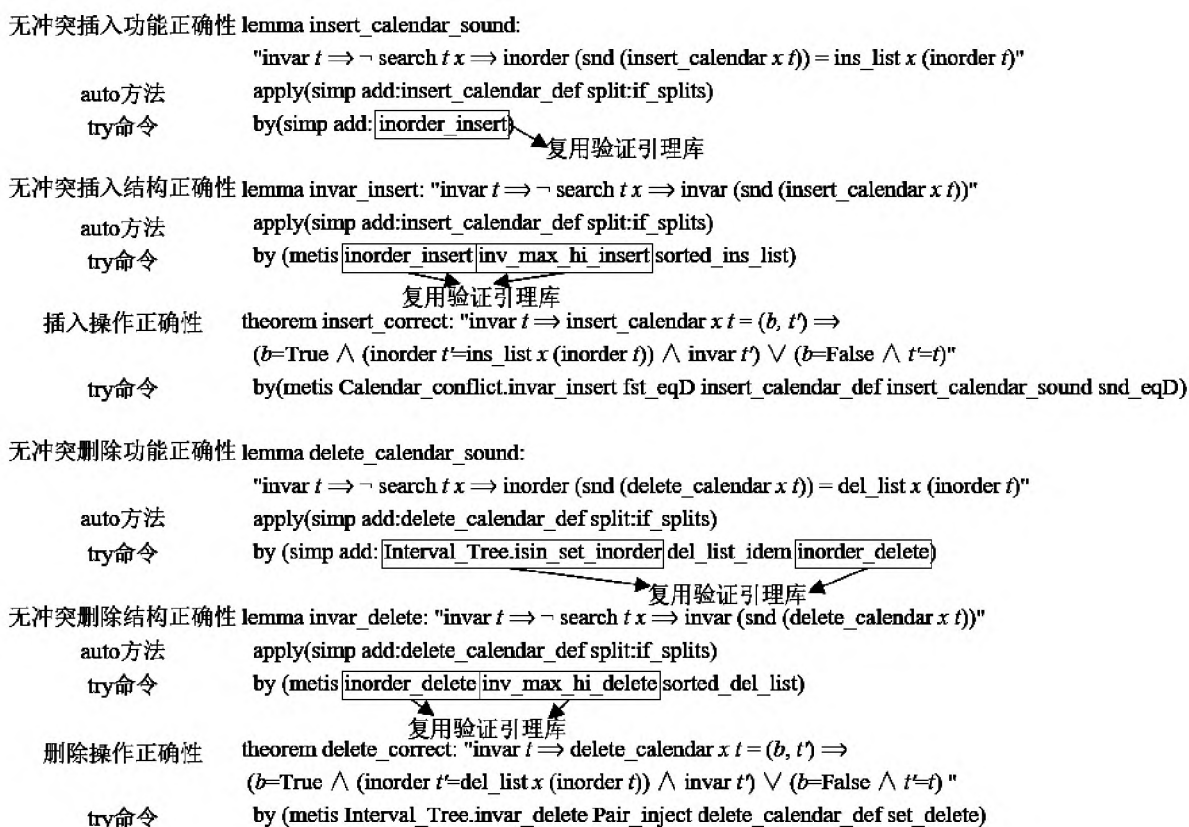


图4 日程冲突检测算法自动化验证过程

用价值.

在未来的研究工作中, 可将基于区间树的建模与验证方法推广至多资源调度和区间优化等更复杂的时间区间类算法, 以提升方法的通用性与适用范围. 此外, 还可考虑结合代码生成机制, 探索由形式化验证模型自动生成高可信可执行程序的方法, 推动验证成果向工程应用转化.

4 总结与展望

本文围绕日程冲突检测算法的形式化建模与自动化验证展开研究, 针对在会议管理、教学排课及医疗排班等典型应用场景中的时间区间冲突问题, 以区间树为基础构建了包含查询类、转换类、修改类及不变式函数的函数式建模库, 并在 Isabelle/HOL 环境下对区间树及其核心操作进行了严格的形式化定义与正确性验证. 在此基础上, 本文设计并实现了基于区间树的日程冲突检测算法, 并通过构建的验证引理库实现了底层操作性质的复用, 从而显著地降低了算法级验证的复杂度. 形式化验证实验以该算法为实例, 对插入与删除关键操作的功能正确性和不变式保持性进行了系统验证. 实验结果表明: 在引理库支持下, 算法核心性质可借助 Isabelle/HOL 的自动化证明工具在较少人工干预的情况下完成验证, 这充分体现了所提出方法在保障算法正确性、提升验证自动化程度方面的有效性与实

5 参考文献

- [1] 陈舒, 陈冷, 陈愉. 基于充电优先级确定机制的电动汽车充电调度策略 [J]. 福建师范大学学报(自然科学版), 2026, 42(1): 10-19.
- [2] PAULSON L C. Computational logic: its origins and applications [J]. Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences, 2018, 474(2210): 20170872.
- [3] 江南, 李清安, 汪吕蒙, 等. 机械化定理证明研究综述 [J]. 软件学报, 2020, 31(1): 82-112.
- [4] NIPKOW T, KLEIN G. Concrete semantics: with Isabelle/

- HOL [M]. Berlin: Springer, 2014.
- [5] NIPKOW T, PAULSON L C, WENZEL M. Isabelle/HOL: a proof assistant for higher-order logic [M]. Berlin: Springer, 2002.
- [6] BÖHME S, NIPKOW T. Sledgehammer: judgement day [EB/OL]. [2025-03-11]. https://link.springer.com/chapter/10.1007/978-3-642-44203-1_9.
- [7] HAO Qi, SHEN Weiming, XUE Yunjiao, et al. Task network-based project dynamic scheduling and schedule coordination [J]. Advanced Engineering Informatics, 2010, 24(4): 417-427.
- [8] XU Lei, CHEN Cailian, ZHANG Yanzhou, et al. Scalable scheduling in time-sensitive networking: an efficient stream conflict detection method [J]. IEEE Transactions on Industrial Informatics, 2025, 21(5): 4105-4116.
- [9] HALLDÓRSSON M M, PATT-SHAMIR B, RAWITZ D. Online scheduling with interval conflicts [J]. Theory of Computing Systems, 2013, 53(2): 300-317.
- [10] WANG Xue, ZHANG Hao, WU Kaijun. Expression tree-based policy conflict detection algorithm [EB/OL]. [2025-03-12]. <https://ieeexplore.ieee.org/abstract/document/9634954>.
- [11] BERRY P M, ALBRIGHT C, BOWRING E, et al. Conflict negotiation among personal calendar agents [EB/OL]. [2025-03-15]. <https://dl.acm.org/doi/abs/10.1145/1160633.1160918>.
- [12] BALLARIN C. Tutorial to locales and locale interpretation [EB/OL]. [2025-03-16]. <https://mediatum.ub.tum.de/1094345>.
- [13] CIGNONI P, MARINO P, MONTANI C, et al. Speeding up isosurface extraction using interval trees [J]. IEEE Transactions on Visualization and Computer Graphics, 1997, 3(2): 158-170.
- [14] MEHTA D P, SAHNI S. Handbook of data structures and applications [M]. New York: Chapman & Hall/CRC, 2004.
- [15] NIPKOW T, BLANCHETTE J, EBERL M, et al. Functional algorithms, verified [EB/OL]. [2025-10-14]. <https://functional-algorithms-verified.org>.
- [16] DROUIN A, HOCKING T D, LAVIOLETTE F. Maximum margin interval trees [EB/OL]. [2025-02-19]. <https://proceedings.cc/paper/2017/hash/2288f691b58edecadccqa-869176264fd-abstract.html>.
- [17] ZHAN Bohua. Verifying imperative programs using auto2 [EB/OL]. [2025-10-14]. https://www.isa-afp.org/browser_info/current/AFP/Auto2_Imperative_HOL/document.pdf.
- [18] BACK R J. Invariant based programming: basic approach and teaching experiences [J]. Formal Aspects of Computing, 2009, 21(3): 227-244.
- [19] 赵永望. 函数式程序设计与证明: Isabelle/HOL 的实践 [EB/OL]. [2025-10-14]. <https://www.yuque.com/zhaoyongwang/fpp>.

The Modeling and Automated Verification Method for Schedule Conflict Detection Algorithm

ZHU Xiaoying, ZUO Zhengkang*

(School of Artificial Intelligence, Jiangxi Normal University, Nanchang Jiangxi 330022, China)

Abstract: Schedule conflict detection algorithm plays an important role in scenarios such as meeting management, course scheduling, and medical duty planning, where the core problem can be abstracted as the detection of overlapping time intervals. As a typical interval query data structure, the interval tree efficiently supports insertion, deletion, and query operations, making it an ideal choice for implementing such algorithms. To ensure the reliability and scalability of the algorithm, a functional modeling library based on interval trees is constructed, together with a supporting verification lemma library, to guarantee the correctness of basic operations under rigorous formal semantics. On this basis, formal modeling and automated verification experiments of the schedule conflict detection algorithm are further carried out. With the support of the modeling library and the verification lemma library, both the functional and structural correctness of the algorithm can be verified using automated proof tools, thereby effectively reducing the cost of manual proofs and demonstrating the feasibility and effectiveness of the proposed method.

Keywords: schedule conflict detection; interval tree; Isabelle theorem prover; functional modeling; automated verification

(责任编辑: 冉小晓)